FILE TANSFER APPLICATIONS USING SOCKETS

REPORT SUBMITTED BY

ACHUTHRAJ R
(CB.SC.U4AIE23108)

ARUNODAY A K
(CB.SC.U4AIE23106)

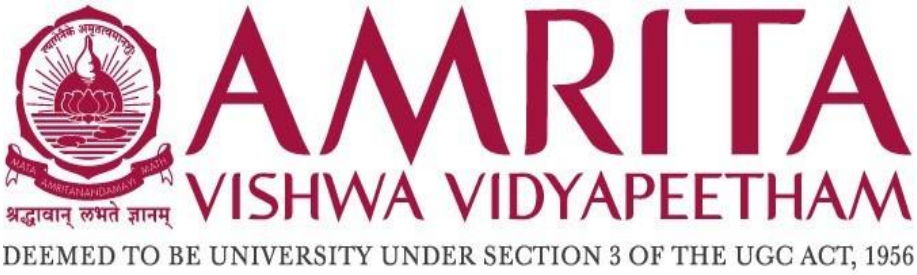
DANDA LOKESH
(CB.SC.U4AIE23123)

RAHUL.B

(CB.SC.U4AIE23114)

AS A PART OF SUBJECT

22AIE204 INTRODUCTION TO COMPUTER NETWORKS



CENTRE FOR COMPUTATIONAL ENGINEERING AND NETWORKING

AMRITA SCHOOLOF ARTIFICIAL INTELLIGENCE

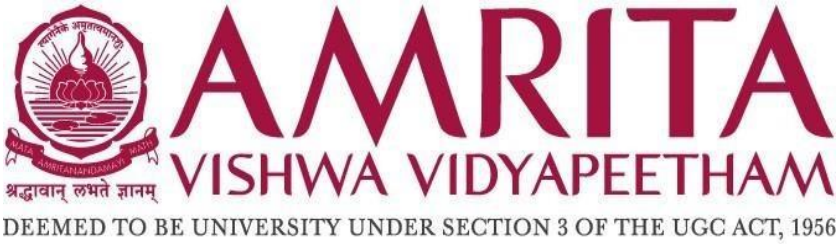
AMRITA VISHWA VIDYAPEETHAM

COIMBATORE-641112(INDIA)

AMRITA SCHOOLOF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE-641112



BONAFIDE CERTIFICATE

This is to certify that the report entitled "FILE TRANSFER APPLICATIONS USING SOCKETS" submitted by ACHUTHRAJ R(CB.SC.U4AIE23108), ARUNODAY A K (CB.SC.U4AIE23106), DANDA LOKESH(CB.SC.U4AIE23123), RAHUL.B (CB.SC.U4AIE23114), for the award of the Degree of Bachelor of Technology in the "CSE(AI)" is a bonafide record of the work carried out by her under our guidance and supervision at Amrita School of Artificial Intelligence, Coimbatore.

Mrs. Ganga Gowri

Project Guide

Dr.K.P.Soman

Professor and Head CEN

Submitted for the university examination held on 11-11-2024

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

DECLARATION

We hereby declare that this thesis entitled “NETWORK PACKET SNIFFER ANALYSER”, is the record of the original work done by me under the guidance of Mrs. Ganga Gowri, Centre for Computational Engineering and Networking, Amrita School of Artificial Intelligence, Coimbatore. To the best of my knowledge this work has not formed the basis for the award of any degree/diploma/ associate ship/fellowship/or a similar award to any candidate in any University.

Signature of the Faculty

Names	**Roll Numbers**
ACHUTHRAJ	**CB.SC.U4AIE23108**
ARUNODAY	**CB.SC.U4AIE23106**
DANDA LOKESH	**CB.SC.U4AIE23123**
RAHUL.B	**CB.SC.U4AIE23114**

Place: Ettimadai
Date:11-11-2024

Table of Contents

1. Abstract……06

2. Introduction……...07

2.1 Overview……07

2.2 Summary of references materials……07

2.3 Objective of the Project……...……08

3. Implementation……09

3.1. Tools and Libraries……09

3.1.1. socket……09

3.1.2. threading……09

3.1.3. os……09

3.2. Client-Server Application……..……..08

3.2.1. Server Architecture……08

3.2.2. Client Implementation……..……10

3.3. Peer-to-Peer Transfer……..……..10

3.3.1. Peer Architecture……...11

3.3.2. File Transfer Implementation……11

3.3.3. Connection Management……..11

3.3.4. Security Considerations……11

4. Results……12

5.Conclusion…….....14

6.References……..15

LIST OF FIGURES

Fig.1: AFTER RUNNING THE SERVER AND CLIEN.T……..… 12

Fig.2: AFTER DOWNLOAD REQUEST…….…….……....…..12

Fig.3: AFTER UPLOAD REQUEST ……..…..… 12

Fig.4: AFTER DISCONNECT REQUEST ……..…..… 13

Fig.5: AFTER ESTABLISHMENT WITH PEER…..……..…..… 13

Fig.6: AFTER UPLOADREQUEST …….……..…..… 13

Fig.7: AFTER DOWNLOAD REQUEST…….…..……..13

Fig.8: AFTER DISCONNECT REQUEST …….…… 13

Abstract

This project presents the design and implementation of two file transfer applications using socket programming, each showcasing different methods for efficient data sharing. The first application is a basic file transfer system that establishes a direct client-server connection, allowing files to be shared quickly and reliably. The second application focuses on peer-to-peer file transfer, enabling data exchange directly between multiple devices without the need for a centralized server.

Using socket programming, both applications support robust connections, handle file segmentation, and allow for concurrent transfers, optimizing performance and reliability. This report details the development process, the network protocols involved, and the testing outcomes, which highlight the practicality of socket-based file transfers for diverse networking scenarios. Overall, the project demonstrates the flexibility and effectiveness of socket programming in building foundational file transfer applications suitable for various real-world environments.

Introduction

2.1 Overview

In computer networking, efficient data transfer and reliable communication are crucial for building systems that can scale and adapt to various demands. This project focuses on developing two distinct file transfer applications using socket programming, which provide different solutions for file sharing within a network. The first approach, a **client-server model**, enables controlled file transfer directly between a client and a server, offering stability and clear management of data flow. The second approach, a **peer-to-peer (P2P) model**, supports file sharing between multiple devices, or "peers," without a central server, creating a more decentralized network. Both models demonstrate the versatility and effectiveness of socket programming in facilitating communication and data exchange across networked environments.

2.2 Summary of Reference Materials

To understand and design the file transfer models in this project, we explored various resources that clarified the workings of both peer-to-peer (P2P) and client-server architectures. The GeeksforGeeks article "P2P (Peer To Peer) File Sharing" provided a comprehensive overview of P2P file sharing, illustrating how files can be shared directly between users without requiring a central server, a key concept in building a decentralized file-sharing system. For the client-server model, the GeeksforGeeks article "Transfer the File 'Client Socket to Server Socket' in Java" offered insights into structured client-server file transfer setups, covering essential steps for sending and receiving files.

Python's official documentation and online tutorials were valuable for implementing socket, threading, and os libraries, which enabled connection management, file segmentation, and concurrent transfers. Network simulations conducted in Cisco Packet Tracer also helped us visualize data flow, optimizing both models for efficient performance.

We also referenced the GitHub repository "Python-Sockets-File-Transfer" by justEhmadSaeed, which showcases a client-server file transfer system implemented with Python sockets Altogether,

these resources offered the technical foundation and real-world examples necessary for developing robust and efficient file-sharing applications.

2.3 Objective of the Project

- Highlight each model's effectiveness in different networking contexts.
- Enhance transfer efficiency through file segmentation.
- Demonstrate socket programming as a foundation for adaptable, real-world networking solutions.

Together, these applications illustrate practical solutions for file sharing in networked environments, offering flexibility, reliability, and efficiency in data communication.

Implementation

3.1. Tools and Libraries:

The applications are implemented in Python, chosen for its simplicity and the rich support it offers for network programming. Python's standard libraries, especially socket, make it a suitable choice for establishing and managing network connections efficiently.

3.1.1. socket: Python's built-in socket library is used for network communication, enabling the application to establish both client-server and peer-to-peer connections. It facilitates direct file transfers by providing reliable transport via TCP/IP protocols, ensuring data integrity during transmission. The library also allows handling socket communication easily, making it ideal for creating scalable, networked applications with minimal setup.

3.1.2. threading: The `threading` library was used to allow multiple operations to occur concurrently, supporting simultaneous file transfers and handling multiple requests. By using threads, the application can manage several tasks without blocking other operations, ensuring a responsive experience for the user. This enables efficient communication between peers, even with numerous concurrent connections.

3.1.3. os: The `os` library assists with file handling, such as reading, writing, and segmenting files before transfer. It ensures compatibility across different operating systems by handling file paths and directory creation. Additionally, the library helps with basic file management, like checking for file existence and creating directories, making file operations seamless in the application.

3.2. Client-Server Application:

The client-server implementation follows a traditional architecture where a central server manages file transfers between connected clients. This section details the key components and their implementation.

3.2.1. Server Architecture

The server implementation consists of several key components:

1. **Connection Management**

    - Utilizes socket binding to port 8000 for accepting client connections
    - Implements a multi-threaded approach to handle multiple client connections simultaneously
    - Each client connection is managed in a separate thread using the handle_client function
    - Maintains a count of active connections for monitoring purposes

2. **File Transfer Protocol**

    - Implements a custom protocol for file transfer with defined message headers

- Uses a 64-byte header to specify message length
- Supports two primary operations: UPLOAD and DOWNLOAD
- Implements end-of-file detection using "--END--" marker

3. **File Management**

    - Creates a dedicated "shared_files" directory for storing transferred files
    - Handles file operations with proper error checking
    - Supports concurrent file access from multiple clients

3.2.2. Client Implementation

The client application provides the following functionalities:

1. **Connection Handling**
    - Establishes TCP connection to the server using configurable IP and port
    - Implements reliable connection management with proper error handling
    - Provides disconnect functionality with proper server notification
2. **File Operations**
    - **Upload Functionality**:
        - Reads files in chunks of 1024 bytes for efficient transfer
        - Sends file data with proper header information
        - Implements progress tracking and completion verification
    - **Download Functionality**:
        - Receives files in chunks to handle large file transfers
        - Implements error handling for missing files
        - Verifies file integrity during transfer
3. **User Interface**
    - Provides a command-line interface for user interaction
    - Supports three main commands: UPLOAD, DOWNLOAD, and DISCONNECT
    - Implements input validation and error handling

3.3. Peer-to-Peer Transfer:

The peer-to-peer implementation allows direct file transfer between nodes without a central server, offering more flexibility and scalability.

3.3.1. Peer Architecture

1. **Dual Mode Operation**

    - Each peer operates in both server and client modes simultaneously
    - Server mode runs in a separate thread to handle incoming connections
    - Client mode allows connecting to other peers for file transfer

2. **Network Management**

    - Uses dynamic IP address resolution using socket.gethostname()
    - Implements port 8000 for peer communication

- Supports multiple simultaneous connections through threading

3.3.2. File Transfer Implementation

1. **Transfer Protocol**

    - Implements a robust file transfer protocol similar to client-server
    - Supports bidirectional file transfer between peers

2. **Error Handling and Recovery**

    - Implements comprehensive error checking for network issues
    - Handles connection drops and reconnection attempts
    - Provides file integrity verification

3.3.3. Connection Management

1. **Peer Discovery**

    - Manual peer address input for connection establishment
    - Supports dynamic peer connection and disconnection
    - Maintains active connection status

2. **Session Control**

    - Implements session management for peer connections
    - Provides clean disconnection handling
    - Supports multiple simultaneous transfer sessions

3.3.4. Security Considerations

1. **Data Integrity**

    - Ensures complete file transfer through chunk verification
    - Handles corrupt transfers with proper error reporting

2. **Error Prevention**

    - Implements file existence checking before transfer
    - Provides proper error messages for failed operations
    - Handles network interruptions gracefully

Results

```
[NEW CONNECTION] ('192.168.43.79', 52744) connected.
[ACTIVE CONNECTIONS] 1
```

Fig.1: AFTER RUNNING THE SERVER AND CLIENT

```
[DOWNLOAD] vid.mp4 downloaded successfully.
```

```
Enter command (UPLOAD/DOWNLOAD/DISCONNECT): DOWNLOAD
Enter filename to download: vid.mp4
[DOWNLOAD] vid.mp4 downloaded successfully.
```

Fig.2: AFTER DOWNLOAD REQUEST

```
[UPLOAD] demo.txt uploaded successfully.
```

```
Enter command (UPLOAD/DOWNLOAD/DISCONNECT): UPLOAD
Enter filename to upload: demo.txt
[UPLOAD] demo.txt uploaded successfully.
```

Fig.3: AFTER UPLOAD REQUEST

```
Enter command (UPLOAD/DOWNLOAD/DISCONNECT): DISCONNECT
PS C:\Users\blues\Desktop\_\Computer Networks\CN\project>
```

Fig.4: AFTER DISCONNECT REQUEST

```
[LISTENING] Peer server is listening on 192.168.43.79
Enter mode (CONNECT/DISCONNECT): CONNECT
Enter peer IP: 192.168.43.94
Enter command (UPLOAD/DOWNLOAD/DISCONNECT):
[NEW CONNECTION] ('192.168.43.94', 63451) connected.
[ACTIVE CONNECTIONS] 2
```

Fig.5: AFTER CONNECTION ESTABLISHMENT WITH PEER

```
UPLOAD
Enter filename to upload: demo.txt

[UPLOAD] demo.txt uploaded successfully.
```

Fig.6: AFTER UPLOAD REQUEST TO PEER

```
Enter command (UPLOAD/DOWNLOAD/DISCONNECT): DOWNLOAD
Enter filename to download: 1.ip

[DOWNLOAD] 1.ip downloaded successfully.
```

Fig.7: AFTER DOWNLOAD REQUEST TO PEER

```
Enter command (UPLOAD/DOWNLOAD/DISCONNECT): DISCONNECT
Enter mode (CONNECT/DISCONNECT): DISCONNECT
PS C:\Users\blues\Desktop\_\Computer Networks\CN\project>
```

Fig.8: AFTER DISCONNECT REQUEST TO PEER

Conclusion

This project developed a versatile file transfer system operating in both client-server and peer-to-peer modes. Built using Python's built-in libraries, such as socket, threading, and os, the application enables efficient, straightforward, and reliable file sharing across networks.

Throughout development and testing, the application consistently handled real-time file transfers, supported multiple concurrent connections, and maintained data integrity. Segmenting files not only improved transfer speeds but also ensured the secure, intact arrival of files. The use of threading enabled the application to manage multiple requests seamlessly, making it responsive and efficient, even under heavy load.

In conclusion, this project demonstrated Python’s adaptability for building networked applications. By implementing both client-server and peer-to-peer models, the system has proven effective for reliable, real-time file sharing across network environments. With optimizations for handling larger files and scalability for additional connections, this application has strong potential as a robust tool for efficient file sharing in diverse networking scenarios.

References

GeeksforGeeks: P2P (Peer-to-Peer) File Sharing:
This article provided an essential understanding of peer-to-peer file-sharing concepts, which were foundational for developing the P2P model in this project.

GeeksforGeeks: Transfer the File "Client Socket to Server Socket" in Java:
This resource offered insights into client-server file transfer setups, covering the processes of sending and receiving files between a client and server.

Python Documentation:
The Python official documentation was invaluable for understanding the socket and threading libraries required to implement reliable network communication and manage concurrent connections. The os library documentation was also referenced for handling file operations.

GitHub Repository: Python-Sockets-File-Transfer by justEhmadSaeed
This GitHub repository provided practical guidance on structuring client-server file transfer systems using Python sockets, helping to streamline file transfer operations and enhance network stability. It served as a reference for managing network connections and error handling within the client-server model.

IBM Documentation: Socket Programming
IBM's documentation on socket programming provided a foundational understanding of networking concepts and socket operations, giving valuable context for creating socket-based applications.

Cisco Packet Tracer:
Cisco Packet Tracer was used as a supplementary tool to simulate network conditions and visualize data flows. This helped in testing the client-server and P2P models within a controlled environment, aiding in debugging and optimizing network performance.